HAC: Simple Hybrids, Smooth Schedules — Rethinking Adaptive Fusion for Resilient UAV Swarm Connectivity Recovery

Abstract

Self-healing connectivity recovery for unmanned aerial vehicle (UAV) swarms requires restoring network connectivity rapidly after partial node failures while preserving wide communication coverage. Peng et al. (2026) recently addressed this trade-off with R2C, combining an intra-subnetwork virtual-force expansion module, a subnetwork-level multipartite graph convolution module, and a time-adaptive weighted fusion schedule (AWF), yet bundling these components obscures the independent roles of spatial reasoning granularity and schedule transition steepness. We systematically ablate this design space across decision paradigm, spatial reasoning granularity, and connectivity–coverage trade-off strategy through five-seed controlled experiments. We propose Hybrid Adaptive Convolution (HAC), which fuses a parameter-free virtual-force heuristic branch with a single global graph convolution branch at the velocity level while omitting subnetwork partitioning, convex-hull boundary extraction, and multipartite graph construction. Across random and uniform deployments, HAC boosts the 80%-failure recovery rate by up to 76.4 percentage points (pp) over single-branch learned baselines, executes in 3.1 ms per step, and retains 88.4% to 91.2% of R2C's reported spatial coverage (within 5.9 to 7.9 pp). Furthermore, we show that transplanting R2C's steep adaptive schedule directly into HAC degrades high-damage recovery, whereas widening the co-active residual window via a smooth transition slope restores robustness and maximizes coverage uniformity. Through a 3×3 train–evaluation schedule decoupling study, we prove that steep schedules fail primarily at deployment time by extinguishing the heuristic safety margin before reconnection completes, whereas time-varying training schedules act as fusion-ratio augmentation that cuts cross-seed variance by up to 87%. Finally, a five-seed subnetwork-level hybrid counter-experiment (HSC) demonstrates that schedule smoothing rescues moderate-damage recovery but cannot overcome rigid-translation expressiveness limits under severe fragmentation.

1 Introduction

1.1 Problem and Motivation

Partial node failures fracture swarm topologies and threaten mission continuity. Unmanned aerial vehicle (UAV) swarms provide flexible wireless communication in post-disaster relief, dynamic battlefields, and congested areas (Coletta et al., 2023; Shi et al., 2025; Wang et al., 2022, 2024b). During operation, internal faults or external attacks can disable a subset of UAVs at any time (Wang et al., 2024a; Kurunathan et al., 2023). When substantial node losses occur, the communication graph fractures into disconnected subnetworks, sharply degrading both communication service and spatial coverage across the mission area (Li et al., 2023; Xu et al., 2022; Peng et al., 2022). The self-healing communication connectivity (SCC) problem seeks a velocity control policy that repositions surviving UAVs to restore full graph connectivity rapidly while maximizing spatial coverage.

Connectivity restoration and spatial coverage preservation are in direct physical tension. Pulling surviving UAVs toward the swarm center accelerates reconnection among fragmented components but shrinks the covered footprint, whereas spreading nodes outward preserves area coverage at the risk of slowing or preventing reconnection. Balancing these competing forces becomes acute when high failure ratios leave small, widely separated subnetworks across the mission space.

1.2 Limitations of Existing Approaches

Hand-crafted heuristic rules fail to scale under severe, unstructured failures. Non-learning SCC algorithms either replace critical cut-vertices through cascaded neighbor movements (Akkaya et al., 2009; Mi and Yang, 2015; Zhang et al., 2024) or guide surviving nodes toward predefined rendezvous points using potential fields (Joshi, 2012; Shriwastav, 2018; Chen et al., 2020; Wang et al., 2024c). Although heuristic rules require no training data and preserve local spacing well, their local sensing assumptions break down when severe node losses separate components beyond communication range (Tosun et al., 2024).

Purely learned graph policies suffer from spatial collapse or rigid-body expressiveness limits. Graph neural network (GNN) methods formulate SCC as a learned graph optimization task and reconnect fragmented topologies faster than heuristics (Mou et al., 2022; Peng et al., 2025; Wang et al., 2025b). However, message-passing aggregation biases surviving UAVs to cluster around high-degree hubs, causing severe spatial collapse and coverage loss. Diffusion-based and differential-subgraph convolutions (Lin et al., 2024; Lin and Ding, 2025) encourage broader dispersion, yet struggle to maintain stable layouts without explicit local repulsion.

1.3 Our Approach and Findings

Isolating spatial granularity and schedule steepness reveals what makes hybrid recovery work. Peng et al. (2026) recently introduced R2C, which combines a buffered virtual-force field for intra-subnetwork expansion (ISE) with a multipartite graph convolution module for inter-subnetwork translation (IST) under an adaptive weighted fusion (AWF) schedule. While R2C achieves strong empirical performance, its module-level ablations toggle entire branches on and off without isolating whether the learned component must operate at subnetwork granularity or whether AWF requires its specific steep sigmoid transition. To answer both questions, we introduce **HAC** (Hybrid Adaptive Convolution), a streamlined architecture that fuses R2C's parameter-free virtual-force branch at the velocity level with a single global k-nearest-neighbor (k-NN) graph convolution branch, eliminating subnetwork partitioning, convex-hull boundary extraction, and multipartite virtual edges.

Five-seed experiments across random and uniform deployments show that fixed-weight global hybridization (**HAC-Fixed**) and smooth adaptive hybridization (**HAC-Smooth**) achieve near-perfect connectivity recovery throughmoderate-to-heavy failures and retain the vast majority of R2C's reported coverage under extreme node loss with substantially lower runtime and variance than single-branch policies. Conversely, directly applying R2C's steep adaptive schedule (**HAC-Steep**) severely degrades high-damage recovery. Our five-seed schedule sweep and orthogonal 3×3 train–evaluation decoupling study establish two core insights. First, a steep sigmoid transition compresses the co-active residual window where both branches collaborate, sequentially concatenating heuristic-only stalling with learned-only clustering during long evaluation rollouts. Second, maintaining non-trivial heuristic weighting at deployment time preserves local link-capture and anti-clustering guarantees, while time-varying fusion during training regularizes the learned branch across diverse weighting ratios and sharply reduces cross-seed variance (**HAC-Curr**). Substituting a subnetwork-level rigid-translation branch (**HSC**) confirms that schedule smoothing rescues moderate fragmentation but cannot overcome the lack of node-level velocity expressiveness under extreme node loss.

1.4 Contributions

1. **Three-axis design taxonomy (§3, §4).** We categorize SCC methods along decision paradigm, spatial reasoning granularity, and connectivity–coverage trade-off strategy, and demonstrate across five random seeds that purely global learned policies (GAC-Fixed) suffer from spatial clustering whereas purely subnetwork-level rigid policies (SAC-Fixed) fail deterministically under heavy node loss.
2. **Minimalist global hybrid architecture (§5, §8).** We propose HAC, fusing a parameter-free virtual-force branch with a global k-NN graph convolution branch. Five-seed evaluations across two deployment geometries show that HAC outperforms single-branch learned baselines by wide margins under severe failures while retaining nearly 90% of R2C's published coverage without boundary-hull or multipartite machinery.
3. **Three-zone curriculum theory and 3×3 schedule decoupling (§6, §9).** We prove a local attractive-zone descent guarantee for the heuristic branch, formulate a three-zone rollout decomposition based on the Co-Active Residual Window (CRW), and evaluate a full 3×3 train–evaluation schedule matrix alongside a five-seed subnetwork-level counter-experiment (HSC) to separate deployment safety margins from training-time regularization.
4. **Count-invariant connectivity loss and reproducible benchmark (§5, Appendix A).** We identify and correct a pair-count bias in log-sum-exp soft-minimum connectivity surrogates, enforce full-stack random seeding, and release the complete multi-seed simulation and training suite.

2 Related Work

2.1 Non-Learning Self-Healing Methods

Critical-node compensation scales poorly with failure density. Early topological recovery schemes detect cut-vertices whose removal partitions the network and trigger cascaded neighbor movements to fill the resulting coverage gaps (Abbasi and Younis, 2008; Akkaya et al., 2009; Mi and Yang, 2015; Zhang et al., 2024). While effective for isolated single-node faults, identifying critical nodes and coordinating multi-hop cascades incur high signaling overhead that fails when simultaneous failures eliminate multiple adjacent relays.

Rendezvous-point aggregation struggles under uncertain fragmentation. Alternative heuristic methods guide surviving UAVs or disconnected clusters toward predetermined rendezvous coordinates using Coulomb-force models (Joshi, 2012), round-table negotiation (Shriwastav, 2018), or artificial potential fields (Chen et al., 2020; Wang et al., 2024c). Although parameter-free and computationally lightweight, fixed attraction rules cannot adapt when surviving subnetworks are separated beyond their local communication horizon (Tosun et al., 2024).

2.2 Learning-Based Self-Healing Methods

Relaxed GNN and attention policies accelerate reconnection but induce spatial collapse. Casting SCC as a differentiable graph optimization problem enables data-driven policies to reason over long-range topological structure (Karalias and Loukas, 2020; Peng et al., 2025). Mou et al. (2022) apply a relaxed graph convolutional network over virtual links, and Guo et al. (2024) incorporate spatiotemporal multi-head attention. Despite faster reconnection than heuristics, message aggregation pulls surviving UAVs tightly toward high-degree hubs, causing severe spatial clustering and coverage loss.

Diffusion and differential-subgraph convolutions lack explicit local collision buffering. Recent learning architectures introduce multi-hop diffusion (Lin et al., 2024) or bipartite convolution over differential subgraphs (Lin and Ding, 2025) to promote dispersed recovery. While these designs improve post-recovery spread compared to standard GCNs, they do not enforce hard inter-agent spacing zones during transit.

2.3 Hybrid Heuristic–Learning Architectures and Positioning

Prior hybrid frameworks either fix node-level weights or bundle subnetwork modules with steep schedules. Li et al. (2025) combine distributed multi-agent reinforcement learning with an artificial potential field at node-level granularity under fixed weighting, whereas Wang et al. (2025b) rely solely on graph reinforcement learning. Related multi-robot coverage and spectral GNN studies explore substructure-aware message passing (Kipf and Welling, 2016; Zeng et al., 2023; Liu and Wang, 2025; Mutzari et al., 2025; Ye et al., 2022). Peng et al. (2026) advance this line with R2C, pairing buffered virtual forces for intra-subnetwork expansion (ISE) with convex-hull boundary extraction (Barber et al., 1996) and Chebyshev multipartite graph convolution for inter-subnetwork translation (IST), blended via a sigmoid adaptive weighted fusion (AWF) schedule (Elfwing et al., 2018). Because R2C's ablations only remove entire modules, the literature lacks a controlled isolation of spatial reasoning granularity and schedule transition steepness. Our work directly bridges this gap.

3 Preliminaries and Problem Formulation

3.1 UAV Swarm Network Model

Swarm connectivity is governed by the algebraic multiplicity of the communication Laplacian. Following Peng et al. (2026), at discrete time step t the surviving UAV swarm forms a temporal proximity graph $G_t = (V_t, E_t, P_t)$, where $V_t = \{v_{1,t}, \ldots, v_{|V_t|,t}\}$ denotes the surviving UAVs, $P_t = \{\mathbf{p}_{1,t}, \ldots, \mathbf{p}_{|V_t|,t}\}$ with $\mathbf{p}_{i,t} \in \mathcal{A} \subset \mathbb{R}^2$ denotes their planar coordinates within mission area $\mathcal{A}$, and E_t denotes active communication links. A direct air-to-air link (Ma et al., 2019; Wang et al., 2025a) exists between v_i and v_j if and only if their Euclidean distance $d_{ij,t} = \|\mathbf{p}_{i,t} - \mathbf{p}_{j,t}\|_2$ satisfies $d_{ij,t} \le R_c$, where the communication radius R_c follows a free-space path-loss model (Ni et al., 2024). Let $\mathbf{A}_t$ be the adjacency matrix and $\mathbf{L}_t = \mathbf{D}_t - \mathbf{A}_t$ be the graph Laplacian. The number of connected components C_t equals the multiplicity of the zero eigenvalue of $\mathbf{L}_t$, so the swarm is fully connected if and only if $C_t = 1$. Each UAV also covers a disk $\mathcal{C}_i(t)$ of radius R_{cov}, yielding the collective area coverage ratio $C_{\text{cov}}(t) = |\bigcup_{i \in V_t} \mathcal{C}_i(t)|/|\mathcal{A}|$.

3.2 Problem Definition

The SCC objective jointly penalizes normalized reconnection time and post-recovery coverage loss. Let t_1 denote the instant of node failure and $t_r \le t_1 + T$ denote the earliest subsequent step at which $C_{t_r} = 1$ within horizon T. The normalized recovery time ratio is $T_{\text{rec}} = (t_r - t_1)/T$. The control policy generates bounded per-node velocities $\hat{V}_t = \{\hat{\mathbf{v}}_{1,t}, \ldots, \hat{\mathbf{v}}_{|V_t|,t}\}$ to minimize

$$J_c = T_{\text{rec}} + \big(1 - C_{\text{cov}}(t_r)\big), \tag{1}$$

subject to four operational constraints: (C1) terminal connectivity $C_{t_r} = 1$ within horizon T, where trials failing to reconnect by $t_1 + T$ are marked unsuccessful and evaluated primarily via recovery success rate; (C2) speed and boundary limits $\|\hat{\mathbf{v}}_{i,t}\|_2 \le \hat{v}_{\max}$ and $\mathbf{p}_{i,t} \in \mathcal{A}$; (C3) discrete kinematics $\mathbf{p}_{i,t+1} = \mathbf{p}_{i,t} + \Delta t \cdot \hat{\mathbf{v}}_{i,t}$; and (C4) link feasibility $d_{ij,t} \le R_c$. Minimizing T_{rec} pulls components inward whereas minimizing $1 - C_{\text{cov}}(t_r)$ pushes nodes outward.

Table 1: Taxonomy of the 3×3×3 SCC design space positioning prior methods and our four controlled architectures (GAC, SAC, HAC, and HSC).

Paradigm	Granularity	Trade-off	Representative method(s)
Heuristic	Node-level	Fixed	Akkaya et al. (2009); Mi and Yang (2015); Wang et al. (2024c)
Heuristic	Subnetwork	Fixed	Chen et al. (2020)
Heuristic	Global	Fixed	Lin and Ding (2025)
Heuristic	Global	Staged	HERO (Mi et al., 2011)
Learned	Node-level	Fixed	Mou et al. (2022)
Learned	Global	Fixed	Guo et al. (2024); **GAC-Fixed (this work, §4)**
Learned	Subnetwork	Fixed	Lin et al. (2024); **SAC-Fixed (this work, §4)**
Hybrid	Node-level	Fixed	Li et al. (2025)
Hybrid	Node+Subnet	Adaptive	R2C (ISE+IST+AWF) (Peng et al., 2026)
Hybrid	**Global**	**Fixed / Adaptive**	**HAC-Fixed / HAC-Smooth / HAC-Curr (this work, §5)**
Hybrid	**Subnetwork**	**Fixed / Adaptive**	**HSC-Fixed / HSC-Smooth (this work, §9)**

3.3 A Taxonomy of the Design Space

Three orthogonal design axes organize prior SCC methods and our controlled ablations. Every self-healing controller specifies three architectural choices:

- **Axis 1 (Decision Paradigm):** Whether motion commands originate from parameter-free physical rules (*heuristic*), a trained parametric network (*learned*), or a velocity-level blend of both (*hybrid*).
- **Axis 2 (Spatial Reasoning Granularity):** Whether the controller reasons over 1-hop communication neighbors (*node-level*), treats each connected component as a rigid unit (*subnetwork-level*), or passes messages across the entire surviving swarm (*global-level*).
- **Axis 3 (Trade-off Strategy):** Whether connectivity and coverage priorities are combined with constant weights (*fixed*), executed in strict temporal phases (*staged*), or continuously reweighted over time (*adaptive*).

Table 1 maps representative literature and the four experimental architectures studied in this work into this $3 \times 3 \times 3$ taxonomy: **GAC-Fixed** (global learned policy), **SAC-Fixed** (subnetwork-level rigid-translation learned policy), **HAC** (our proposed hybrid combining node-level heuristic forces with a global learned GCN under fixed, smooth adaptive, or curriculum-decoupled schedules), and **HSC** (a hybrid counter-experiment pairing the same heuristic branch with SAC's subnetwork-level learned branch).

4 Motivating Experiments: Failure Modes of Single-Branch Designs

Single-branch learned policies face an architectural trade-off between spatial collapse and fragmentation failure. Before introducing our hybrid architecture, we evaluate two pure learned baselines that isolate the spatial granularity axis under identical loss functions and training rollouts:

- **Global Adaptive-weighted Convolution (GAC-Fixed):** A 4-layer GCN operating over a Euclidean k-NN graph ($k = 8$) across all surviving UAVs regardless of communication range R_c, predicting bounded per-node 2D velocities directly via tanh scaling.

Table 2: Motivating single-branch learned baselines under random deployment (mean ± std over 5 seeds, 50 trials per seed). GAC-Fixed exhibits spatial clustering (elevated AND), whereas SAC-Fixed avoids clustering but fails deterministically at 60%–80% node loss.

Failure	GAC-Fixed (global learned)		SAC-Fixed (subnetwork learned)	
rate	AND	Success/50	AND	Success/50
20%	13.68 ± 0.28	49.0 ± 0.6	6.71 ± 0.09	29.4 ± 3.4
40%	12.48 ± 0.61	47.2 ± 1.7	5.32 ± 0.18	8.8 ± 1.9
60%	10.55 ± 0.78	42.8 ± 3.7	n/a	0.0 ± 0.0
80%	6.00 ± 0.26	19.8 ± 5.5	n/a	0.0 ± 0.0

- **Subnetwork-level Adaptive Convolution (SAC-Fixed):** A 4-layer GCN operating over a k-NN graph of connected-component centroids (pooled from component coordinates and relative sizes), predicting a single rigid 2D translation velocity shared by all UAVs within each subnetwork.

Global message passing causes severe spatial clustering, whereas rigid subnetwork translation collapses under moderate-to-high node loss. Table 2 summarizes five-seed evaluations under random deployment. GAC-Fixed maintains moderate connectivity recovery up to 60% node failure but produces an elevated trajectory-averaged node degree (AND of 13.68 ± 0.28 at 20% failure, more than double SAC-Fixed's 6.71 ± 0.09), confirming that unconstrained global GNN aggregation pulls UAVs into dense clusters at the expense of area coverage. Conversely, forcing all nodes within a component to translate rigidly in SAC-Fixed prevents internal clustering and keeps AND low, but destroys recovery capability as fragmentation increases, dropping to 8.8 ± 1.9 recoveries out of 50 at 40% failure and 0.0 ± 0.0 across all five seeds at 60% and 80% failure. Because isolated single-UAV and small-cluster subnetworks cannot reshape their internal boundaries to bridge wide gaps under rigid translation, neither granularity succeeds alone. This complementary failure pattern directly motivates fusing a local spacing heuristic with a global learned velocity field.

5 Method: Hybrid Adaptive Convolution (HAC)

5.1 Overview and Architectural Minimalism

HAC pairs parameter-free local virtual forces with a single global graph convolution at the velocity level. To test whether R2C's complex subnetwork machinery is necessary, HAC instantiates the *hybrid* × *global* cell of Table 1 in its simplest form. Each surviving UAV's velocity command is synthesized from two parallel branches: a deterministic virtual-force heuristic branch that regulates 1-hop neighbor spacing and boundary avoidance, and a global k-NN GCN branch that provides long-range directional guidance across disconnected components without convex-hull boundary extraction or multipartite virtual edges.

5.2 Heuristic Branch: Buffered Virtual-Force Field

A three-zone virtual spring regulates local spacing and prevents boundary escape without learnable parameters. Adopting R2C's buffered Hookean force model (Rychlewski, 1984; Peng et al., 2026) with neutral zone $[d_l, d_u] = [0.8R_c, 0.9R_c] \subset (0, R_c]$, the pairwise force exerted on UAV

v_i by a communication neighbor $v_j \in \mathcal{N}_i = \{j : 0 < d_{ij,t} \le R_c\}$ is

$$\mathbf{F}'_{ij,t} = \begin{cases} -\exp\left(-\alpha \cdot \dfrac{d_{ij,t}}{d_l}\right) \mathbf{u}_{ij,t}, & 0 < d_{ij,t} < d_l, \\ \mathbf{0}, & d_l \le d_{ij,t} \le d_u, \\ \exp\left(-\beta \cdot \dfrac{R_c - d_{ij,t}}{R_c - d_u}\right) \mathbf{u}_{ij,t}, & d_u < d_{ij,t} \le R_c, \end{cases} \tag{2}$$

where $\mathbf{u}_{ij,t} = (\mathbf{p}_{j,t} - \mathbf{p}_{i,t})/d_{ij,t}$ is the unit vector pointing from v_i to v_j, and $\alpha = \beta = 0.01$. To prevent UAVs from drifting outside the mission square $\mathcal{A} = [0, L]^2$, any UAV within buffer distance $d_e = 0.15R_c$ of a boundary edge $b \in \mathcal{B} = \{\text{left, right, bottom, top}\}$ receives an inward repulsion force $\mathbf{F}''_{i,t} = \sum_{b\in\mathcal{B},\, 0\le d_{i,b}<d_e} \exp(-\delta \cdot d_{i,b})\, \mathbf{n}_b$, where $d_{i,b} \ge 0$ is the perpendicular distance to edge b, $\mathbf{n}_b$ is its inward unit normal, and $\delta = 0.01$. Summing over communication neighbors yields $\mathbf{F}_{i,t} = \sum_{j\in\mathcal{N}_i} \mathbf{F}'_{ij,t} + \mathbf{F}''_{i,t}$ and normalized heuristic velocity $\mathbf{v}'_{i,t} = \hat{v}_{\max} \cdot \mathbf{F}_{i,t}/(\|\mathbf{F}_{i,t}\|_2 + \epsilon)$.

5.3 Learned Branch: Global Graph Convolution

A global Euclidean k-NN graph enables cross-component message passing without subnetwork partitioning. At each step t, we construct a symmetric k-NN graph ($k = 8$) over the Euclidean coordinates of all surviving UAVs so that topological information propagates across disconnected components even when inter-component distances exceed R_c. Each UAV v_i is represented by a 3-dimensional feature vector $\mathbf{x}_{i,t} = [p_{i,x}/L,\, p_{i,y}/L,\, \deg_k(v_i)/\max_j \deg_k(v_j)]^\top$, omitting explicit subnetwork IDs so that node representations remain invariant to arbitrary component labelings. Passing $\{\mathbf{x}_{i,t}\}$ through an $L_{\text{gcn}} = 4$ layer GCN (Kipf and Welling, 2016) with hidden dimension 128 and ReLU activations, followed by a two-layer MLP and elementwise tanh scaling by $\hat{v}_{\max}$, produces the learned velocity vector $\mathbf{v}''_{i,t}$.

5.4 Velocity-Level Fusion Modes

Velocity-level normalization blends local heuristic expansion with global learned translation under fixed, smooth, or decoupled schedules. Given time horizon H ($H = T_{\text{train}}$ during training and $H = T$ during evaluation), the adaptive fusion schedule assigns weights $w_T(t)$ and $w_E(t)$ via a parameterized sigmoid:

$$w_T(t; k, s_0) = \frac{1}{1 + \exp\big(-k \cdot (t/H - s_0)\big)}, \qquad w_E(t; k, s_0) = 1 - w_T(t; k, s_0), \tag{3}$$

where $k > 0$ governs transition steepness and $s_0 \in (0, 1)$ sets the midpoint fraction. The two branch velocities are fused and renormalized to maximum speed $\hat{v}_{\max}$:

$$\hat{\mathbf{v}}_{i,t} = \hat{v}_{\max} \cdot \frac{w_E(t)\, \mathbf{v}'_{i,t} + w_T(t)\, \mathbf{v}''_{i,t}}{\left\| w_E(t)\, \mathbf{v}'_{i,t} + w_T(t)\, \mathbf{v}''_{i,t} \right\|_2 + \epsilon}. \tag{4}$$

We define four concrete scheduling modes across training (w_T^{train}) and evaluation (w_T^{eval}): (i) **HAC-Fixed** sets $w_E = w_T = 0.5$ during both training and evaluation; (ii) **HAC-Steep** uses R2C's steep default ($k = 30, s_0 = 0.125$) throughout; (iii) **HAC-Smooth** uses a gentle transition ($k = 5, s_0 = 0.7$) throughout; and (iv) **HAC-Curr** trains under the smooth schedule ($k = 5, s_0 = 0.7$) and evaluates with fixed equal weights $w_E = w_T = 0.5$.

Algorithm 1 HAC End-to-End Differentiable Rollout Training (one epoch)

Require: Learned policy π_θ, rollout horizon T_{train}, training schedule $w_T^{\text{train}}(\cdot)$

Sample failure ratio $r \in \{0.2, 0.4, 0.6, 0.8\}$; initialize surviving positions $\mathbf{P}_0$
$\mathcal{L}_{\text{total}} \leftarrow 0$
for $t = 0, \ldots, T_{\text{train}} - 1$ **do**
  Partition $\mathbf{P}_t$ into connected components $\{S_m\}$ under communication range R_c
  Compute parameter-free heuristic velocities $\mathbf{v}'_t$ via Eq. (2) ▷ stop gradient
  Construct global Euclidean k-NN graph on $\mathbf{P}_t$ and predict $\mathbf{v}''_t \leftarrow \hat{v}_{\max}\, \pi_\theta(\mathbf{P}_t)$
  Compute fused velocities $\hat{\mathbf{v}}_t$ from $(\mathbf{v}'_t, \mathbf{v}''_t, 1 - w_T^{\text{train}}(t), w_T^{\text{train}}(t))$ via Eq. (4)
  $\mathbf{P}_{t+1} \leftarrow \text{clip}(\mathbf{P}_t + \Delta t \cdot \hat{\mathbf{v}}_t,\ 0,\ L)$
  $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{total}} + 0.5\, \mathcal{L}_{\text{conn}}(\mathbf{P}_{t+1}) + 0.5\, \mathcal{L}_{\text{cov}}(\mathbf{P}_{t+1}) + \lambda\, \mathcal{L}_{\text{move}}(\mathbf{P}_t, \mathbf{P}_{t+1})$
end for
Backpropagate $\mathcal{L}_{\text{total}}/T_{\text{train}}$ and update θ via Adam with gradient clipping at 1.0

Algorithm 2 HAC Deployment Rollout

Require: Trained policy π_θ, evaluation horizon T, evaluation schedule $w_T^{\text{eval}}(\cdot)$

for $t = 0, \ldots, T - 1$ **do**
  if $C_t = 1$ **then return** success at recovery step $t_r = t$
  end if
  Compute $\mathbf{v}'_t$ via Eq. (2) and $\mathbf{v}''_t \leftarrow \hat{v}_{\max}\, \pi_\theta(\mathbf{P}_t)$
  Update $\mathbf{P}_{t+1} \leftarrow \text{clip}\big(\mathbf{P}_t + \Delta t \cdot \text{fuse}(\mathbf{v}'_t, \mathbf{v}''_t, 1 - w_T^{\text{eval}}(t), w_T^{\text{eval}}(t)),\ 0,\ L\big)$
end for
return failure (horizon T exhausted with $C_T > 1$)

5.5 Count-Invariant Differentiable Training Objective

Subtracting log pair counts makes soft-minimum inter-component distance estimation invariant to cluster size. Because the discrete component count C_t is non-differentiable, we train the learned branch end-to-end through T_{train}-step differentiable rollouts using a corrected soft-minimum connectivity surrogate. For every pair of disconnected components (S_i, S_j) at step t with node counts $n_i = |S_i|$ and $n_j = |S_j|$, we define

$$\mathcal{L}_{\text{conn}}^{(i,j)} = -\tau_{\text{c}} \left(\log \sum_{u \in S_i,\, v \in S_j} \exp\left(-\frac{d_{uv,t}}{\tau_{\text{c}}} \right) - \log(n_i n_j) \right), \tag{5}$$

with temperature $\tau_{\text{c}} = 50\,\text{m}$, averaged over all $\binom{C_t}{2}$ component pairs. Without the $-\log(n_i n_j)$ correction, standard log-sum-exp systematically under-penalizes large components by $\tau_{\text{c}} \log(n_i n_j)$ and produces spuriously negative losses (Appendix A). Area coverage is approximated over Monte Carlo grid samples G_{mc} via $\mathcal{L}_{\text{cov}} = \frac{1}{|G_{\text{mc}}|} \sum_{\mathbf{g} \in G_{\text{mc}}} \big(1 - \sigma\big((R_{\text{cov}} - \min_i \|\mathbf{g} - \mathbf{p}_{i,t}\|_2)/\tau_{\text{v}}\big)\big)$ with $\tau_{\text{v}} = 10\,\text{m}$, and movement regularization is $\mathcal{L}_{\text{move}} = \frac{1}{|V_t|} \sum_i \|\mathbf{p}_{i,t+1} - \mathbf{p}_{i,t}\|_2$. The per-step loss is $\mathcal{L}_t = 0.5\, \mathcal{L}_{\text{conn}} + 0.5\, \mathcal{L}_{\text{cov}} + \lambda\, \mathcal{L}_{\text{move}}$ with $\lambda = 0.01$. Algorithms 1 and 2 detail the training and inference procedures.

6 Theoretical Analysis

6.1 Local Monotonicity of the Heuristic Branch

The heuristic virtual-force branch provides a certifiable local descent direction on inter-node distance within the attractive zone. We first analyze the test-time link-capture stability

provided by the parameter-free branch.

Proposition 1 (Attractive-zone descent guarantee)**.** *Let v_i, v_j be two surviving UAVs at distance $d_u < d_{ij,t} \le R_c$ in the attractive zone of Eq. (2). If no repulsive neighbor forces or boundary forces act on v_i and v_j, then applying the heuristic velocity alone ($w_E = 1, w_T = 0$) with step size $0 < \Delta t < d_{ij,t}/(2\hat{v}_{\max})$ strictly decreases their Euclidean separation: $d_{ij,t+1} = d_{ij,t} - 2\Delta t\, \hat{v}_{\max} \frac{\|\mathbf{F}'_{ij,t}\|_2}{\|\mathbf{F}'_{ij,t}\|_2+\epsilon} < d_{ij,t}$.*

Proof. By Eq. (2), $\mathbf{F}'_{ij,t} = m_{ij}\,\mathbf{u}_{ij,t}$ with magnitude $m_{ij} = \exp\!\big(-\beta \frac{R_c - d_{ij,t}}{R_c - d_u}\big) > 0$ along $\mathbf{u}_{ij,t} = (\mathbf{p}_{j,t} - \mathbf{p}_{i,t})/d_{ij,t}$, while by anti-symmetry $\mathbf{F}'_{ji,t} = -m_{ij}\,\mathbf{u}_{ij,t}$. Under the isolation condition, $\mathbf{v}'_{i,t} = \tilde{v}\,\mathbf{u}_{ij,t}$ and $\mathbf{v}'_{j,t} = -\tilde{v}\,\mathbf{u}_{ij,t}$ where $\tilde{v} = \hat{v}_{\max}\frac{m_{ij}}{m_{ij}+\epsilon} \in (0, \hat{v}_{\max})$. Updating positions yields $\mathbf{p}_{i,t+1} - \mathbf{p}_{j,t+1} = (\mathbf{p}_{i,t} - \mathbf{p}_{j,t}) + 2\Delta t\,\tilde{v}\,\mathbf{u}_{ij,t} = -(d_{ij,t} - 2\Delta t\,\tilde{v})\mathbf{u}_{ij,t}$. Taking the Euclidean norm gives $d_{ij,t+1} = d_{ij,t} - 2\Delta t\,\tilde{v} < d_{ij,t}$. □

Remark 1 (Test-time safety margin under branch fusion)**.** *In multi-UAV rollouts, competing neighbor forces and the learned velocity $\mathbf{v}''_{i,t}$ perturb this descent direction. Because the numerator of Eq. (4) is the convex combination $w_E(t)\mathbf{v}'_{i,t} + w_T(t)\mathbf{v}''_{i,t}$ of two vectors bounded in norm by $\hat{v}_{\max}$, preserving a positive contribution from the heuristic branch's attractive lock and repulsive spacing requires keeping $w_E(t)$ above a safety margin $w_E^{\min} \in (0, 0.5]$, or equivalently keeping $w_T(t) \le 1 - w_E^{\min}$. Under a steep schedule ($k = 30, s_0 = 0.125$), $w_E(t)$ falls below 0.01 by $t/H = 0.28$, whereas 80%-failure recovery requires $t_r/T \approx 0.68$. Consequently, for more than half of the evaluation trajectory ($t/T \in [0.28, 0.68]$), the controller degenerates into an unbuffered learned policy that breaks newly captured links and induces spatial clustering.*

6.2 Three-Zone Rollout Decomposition and Co-Active Residual Window

A steep sigmoid schedule compresses the co-active window where both branches collaborate, sequentially concatenating single-branch failure modes. To explain why both early steep transitions ($k = 30, s_0 = 0.125$) and delayed steep transitions ($k = 30, s_0 = 0.5$) fail, we partition any horizon H under schedule $w_T(t; k, s_0)$ into three functional regimes bounded by threshold margin $\delta_w = 0.2$:

1. **Heuristic-Dominated / Gradient-Starvation Zone ($w_T(t) < 0.2$):** The controller behaves nearly as Heuristic-only, which cannot bridge components separated beyond R_c; during training, backpropagation to π_θ is attenuated by factor $w_T(t) < 0.2$.
2. **Co-Active Residual Window (**CRW**, $0.2 \le w_T(t) \le 0.8$):** Both branches contribute substantially, allowing the global GCN to steer disconnected components together while the virtual-force branch enforces local spacing and link capture.
3. **Unassisted High-Autonomy Zone (**AE**, $w_T(t) > 0.8$):** The heuristic safety margin $w_E(t) < 0.2$ vanishes, exposing the swarm to GNN spatial collapse.

Definition 1 (Co-Active Residual Window and High-Autonomy Exposure)**.** *For horizon H, the* Co-Active Residual Window $\mathrm{CRW}(k, s_0)$ *and* High-Autonomy Exposure $\mathrm{AE}(k, s_0)$ *are the fractions of rollout steps falling in $[0.2, 0.8]$ and $(0.8, 1.0]$, respectively:*

$$\mathrm{CRW}(k, s_0) = \frac{1}{H}\Big|\Big\{t : 0.2 \le w_T(t; k, s_0) \le 0.8\Big\}\Big|, \qquad \mathrm{AE}(k, s_0) = \frac{1}{H}\Big|\Big\{t : w_T(t; k, s_0) > 0.8\Big\}\Big|. \quad (6)$$

Inverting Eq. (3) shows that $w_T(t; k, s_0)$ crosses from 0.2 to 0.8 over continuous fractional width

$$\Delta s_{20\to 80} = \frac{\ln(16)}{k} \approx \frac{2.773}{k}, \quad (7)$$

and enters the high-autonomy zone $w_T > 0.8$ at fractional step $s_{0.8} = s_0 + \frac{\ln 4}{k} \approx s_0 + \frac{1.386}{k}$. Equation (7) explains why delaying a steep schedule without reducing k is counterproductive: at $(k = 30, s_0 = 0.5)$, the swarm spends the first 47.5% of the horizon stalled in the Heuristic-Dominated Zone ($w_T < 0.2$), passes through a fleeting Co-Active Residual Window of only CRW = 7.5% ($\Delta s_{20\to 80} = 9.2\%$), and then jumps directly into the Unassisted High-Autonomy Zone (AE = 45.0%). By contrast, reducing k to 5 at $s_0 = 0.7$ (**HAC-Smooth**) expands CRW to 57.5% and eliminates high-autonomy exposure (AE = 0.0%).

7 Experimental Setup

Simulation testbed and failure protocols follow R2C's physical parameters across random and uniform deployments. We evaluate all methods in a $1000 \times 1000\,\mathrm{m}^2$ mission square under two initial swarm layouts: **random deployment** with 200 UAVs sampled uniformly at random, and **uniform deployment** with 196 UAVs placed on a 14×14 regular grid. Following Peng et al. (2026), each UAV has communication and coverage radii $R_c = R_{\mathrm{cov}} = 120\,\mathrm{m}$, maximum speed $\hat{v}_{\max} = 1.0\,\mathrm{m/s}$, step size $\Delta t = 1.0\,\mathrm{s}$, and evaluation horizon $T = 400$ steps. At $t_1 = 0$, a failure ratio $r \in \{20\%, 40\%, 60\%, 80\%\}$ of UAVs is removed uniformly at random. Every architecture and ablation configuration is trained across 5 independent random seeds (`seed` $\in \{1, 2, 3, 4, 5\}$) and evaluated over 50 independent failure trials per seed (250 trials per configuration per failure rate).

Baselines and evaluation metrics quantify recovery reliability, speed, coverage, clustering, and runtime. We compare **HAC-Fixed**, **HAC-Smooth** ($k = 5, s_0 = 0.7$), and **HAC-Curr** against five baselines: (i) **GAC-Fixed** (global learned GCN with direct tanh-bounded velocity output), (ii) **SAC-Fixed** (subnetwork rigid-translation GCN), (iii) **Learned-only** ($w_T(t) \equiv 1.0$ ablation of HAC with unit-norm velocity rescaling), (iv) **Heuristic-only** ($w_E(t) \equiv 1.0$ virtual-force branch), and (v) published reference numbers from **R2C** (Peng et al., 2026). We report five metrics: **Success/50** (trials out of 50 achieving $C_{t_r} = 1$ within $T = 400$ steps), **REC** (recovery time ratio T_{rec} averaged over successful trials), **COV** (coverage ratio $C_{\mathrm{cov}}(t_r)$ at recovery), **AND** (trajectory-averaged node degree up to t_r), and wall-clock rollout runtime.

Implementation details and full-stack seeding ensure statistical reproducibility. All learned backbones use $L_{\mathrm{gcn}} = 4$ GCN layers, hidden width 128, ReLU activations, and a 2-layer MLP head, trained for 300 epochs using Adam (learning rate 10^{-3}, gradient norm clip 1.0) over $T_{\mathrm{train}} = 40$ rollout steps with 800 Monte Carlo coverage grid points per step. Both NumPy environment RNGs and PyTorch CPU/CUDA weight initializations are strictly seeded prior to model instantiation.

8 Results

8.1 Main Results: HAC vs. Single-Branch Baselines

HAC variants achieve near-perfect recovery through 60% failure and outperform single-branch baselines by up to 76.4 pp in recovery rate at 80% node loss. Tables 3 and 4 report the five-seed evaluation results under random and uniform deployments, and Figure 1 (left) illustrates the random-deployment recovery trajectories. Across 20% to 60% failure rates in both geometries, HAC-Fixed achieves a 100% recovery rate (50.0 ± 0.0), while HAC-Smooth achieves faster reconnection at 20%–60% random failure and higher post-recovery coverage (0.949 vs. 0.928 at 60% random failure; 0.967 vs. 0.960 at 60% uniform failure). At the extreme 80% failure rate, HAC-Fixed succeeds in 46.6 ± 3.9 (random) and 49.4 ± 0.8 (uniform) trials, and HAC-Curr reaches 47.4 ± 1.9 (random), surpassing GAC-Fixed by up to 27.6 trials (55.2 pp) and Learned-only by up to 38.2 trials (76.4 pp), whereas SAC-Fixed and Heuristic-only fail completely (0.0 ± 0.0). Notably, Learned-only

Table 3: Main results under random deployment (mean ± std over 5 seeds, 50 evaluation trials per seed). Success is out of 50 trials; Runtime is mean wall-clock rollout time in seconds.

Architecture	Failure	Success/50	REC	COV	AND	Runtime (s)
HAC-Fixed	20%	**50.0 ± 0.0**	0.051 ± 0.011	0.997 ± 0.000	6.73 ± 0.05	0.38 ± 0.04
	40%	**50.0 ± 0.0**	0.134 ± 0.005	0.988 ± 0.001	5.35 ± 0.05	0.51 ± 0.03
	60%	**50.0 ± 0.0**	0.331 ± 0.009	0.928 ± 0.008	4.01 ± 0.07	0.80 ± 0.04
	80%	**46.6 ± 3.9**	0.696 ± 0.048	0.602 ± 0.009	2.58 ± 0.03	**1.28 ± 0.11**
HAC-Smooth (5, 0.7)	20%	49.8 ± 0.4	**0.034 ± 0.008**	0.997 ± 0.000	6.63 ± 0.03	**0.34 ± 0.02**
	40%	**50.0 ± 0.0**	**0.110 ± 0.007**	**0.993 ± 0.000**	5.22 ± 0.04	**0.44 ± 0.02**
	60%	49.6 ± 0.5	**0.325 ± 0.020**	**0.949 ± 0.004**	3.86 ± 0.06	**0.78 ± 0.03**
	80%	43.6 ± 2.8	**0.684 ± 0.020**	**0.616 ± 0.006**	**2.48 ± 0.06**	1.29 ± 0.04
GAC-Fixed	20%	49.0 ± 0.6	0.324 ± 0.007	0.747 ± 0.009	13.68 ± 0.28	1.42 ± 0.09
	40%	47.2 ± 1.7	0.467 ± 0.020	0.614 ± 0.021	12.48 ± 0.61	1.95 ± 0.12
	60%	42.8 ± 3.7	0.612 ± 0.027	0.452 ± 0.022	10.55 ± 0.78	2.18 ± 0.11
	80%	19.8 ± 5.5	0.775 ± 0.017	0.310 ± 0.009	6.00 ± 0.26	1.84 ± 0.08
SAC-Fixed	20%	29.4 ± 3.4	0.232 ± 0.047	0.942 ± 0.014	6.71 ± 0.09	2.24 ± 0.26
	40%	8.8 ± 1.9	0.577 ± 0.068	0.842 ± 0.037	5.32 ± 0.18	2.72 ± 0.21
	60–80%	0.0 ± 0.0	n/a	n/a	n/a	1.99 ± 0.14
Learned-only	20%	45.6 ± 2.9	0.212 ± 0.025	0.901 ± 0.014	9.07 ± 0.44	1.71 ± 0.32
	40%	37.8 ± 3.5	0.468 ± 0.030	0.719 ± 0.025	10.12 ± 0.33	2.59 ± 0.18
	60%	26.4 ± 2.8	0.702 ± 0.020	0.503 ± 0.024	9.27 ± 0.46	2.47 ± 0.08
	80%	8.6 ± 3.3	0.831 ± 0.023	0.347 ± 0.023	5.10 ± 0.33	1.89 ± 0.06
Heuristic-only	20%	47.2 ± 1.2	0.045 ± 0.015	0.997 ± 0.000	6.64 ± 0.04	0.36 ± 0.04
	40%	30.8 ± 1.3	0.125 ± 0.021	0.993 ± 0.001	5.14 ± 0.05	0.63 ± 0.02
	60%	1.4 ± 1.4	0.227 ± 0.092	0.964 ± 0.007	3.78 ± 0.13	0.78 ± 0.03
	80%	0.0 ± 0.0	n/a	n/a	n/a	0.55 ± 0.02

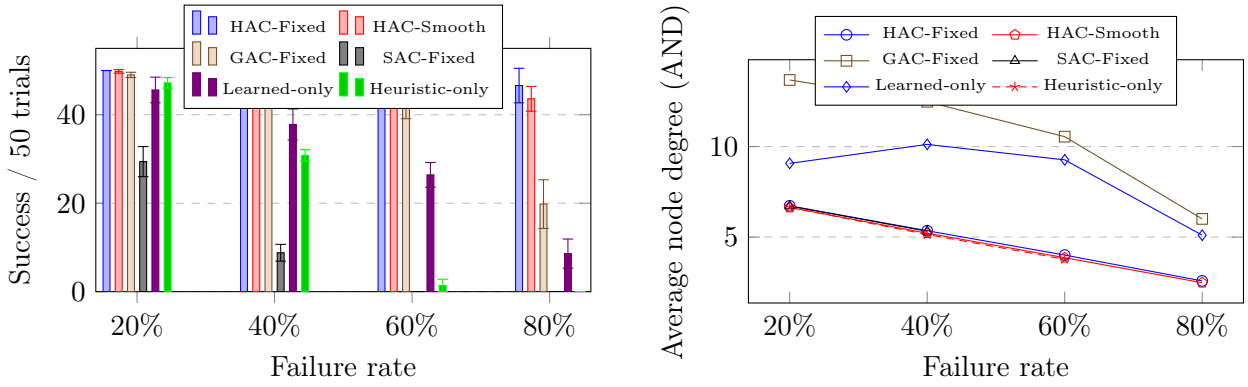


Figure 1: Random-deployment five-seed comparisons across failure rates: (left) connectivity recovery success count out of 50 trials; (right) trajectory-averaged node degree (AND). HAC-Fixed and HAC-Smooth sustain high recovery rates while matching the low clustering profile of non-clustering baselines.

trails GAC-Fixed at 80% random failure (8.6 vs. 19.8) because setting $w_T \equiv 1$ in Eq. (4) forces every UAV to move at constant unit speed $\hat{v}_{\max}$ without heuristic damping, whereas GAC-Fixed modulates speed continuously via tanh.

8.2 Comparison with Published R2C Results and Runtime Efficiency

Without boundary hulls or multipartite graph convolutions, HAC retains 88.4% to 91.2% of R2C's reported coverage at 80% failure (within 5.9 to 7.9 pp) while executing in 3.1 ms per step. Table 5 compares five-seed means of HAC-Fixed and HAC-Smooth against R2C's published

Table 4: Main results under uniform grid deployment (mean ± std over 5 seeds, 50 evaluation trials per seed).

Architecture	Failure	Success/50	REC	COV	AND	Runtime (s)
HAC-Fixed	20%	$\mathbf{50.0 \pm 0.0}$	0.005 ± 0.001	1.000 ± 0.000	5.72 ± 0.01	0.24 ± 0.02
	40%	$\mathbf{50.0 \pm 0.0}$	0.040 ± 0.004	0.998 ± 0.000	4.35 ± 0.02	0.27 ± 0.03
	60%	$\mathbf{50.0 \pm 0.0}$	$\mathbf{0.249 \pm 0.023}$	0.960 ± 0.004	3.30 ± 0.04	0.65 ± 0.04
	80%	$\mathbf{49.4 \pm 0.8}$	$\mathbf{0.643 \pm 0.026}$	$\mathbf{0.609 \pm 0.004}$	2.30 ± 0.03	$\mathbf{1.21 \pm 0.06}$
HAC-Smooth $(5, 0.7)$	20%	$\mathbf{50.0 \pm 0.0}$	0.005 ± 0.001	1.000 ± 0.000	5.72 ± 0.01	$\mathbf{0.23 \pm 0.01}$
	40%	$\mathbf{50.0 \pm 0.0}$	$\mathbf{0.039 \pm 0.004}$	0.998 ± 0.000	4.33 ± 0.02	$\mathbf{0.26 \pm 0.02}$
	60%	$\mathbf{50.0 \pm 0.0}$	0.252 ± 0.019	$\mathbf{0.967 \pm 0.001}$	3.24 ± 0.02	$\mathbf{0.63 \pm 0.04}$
	80%	47.8 ± 1.3	0.665 ± 0.040	0.604 ± 0.007	$\mathbf{2.28 \pm 0.03}$	1.26 ± 0.10
GAC-Fixed	20%	50.0 ± 0.0	0.011 ± 0.003	0.992 ± 0.003	5.88 ± 0.04	0.31 ± 0.03
	40%	50.0 ± 0.0	0.200 ± 0.014	0.843 ± 0.019	6.86 ± 0.18	0.98 ± 0.08
	60%	47.0 ± 2.8	0.574 ± 0.013	0.478 ± 0.023	8.75 ± 0.42	1.92 ± 0.11
	80%	25.8 ± 6.1	0.772 ± 0.057	0.300 ± 0.012	5.81 ± 0.31	1.88 ± 0.09
SAC-Fixed	20%	48.8 ± 1.2	0.007 ± 0.007	0.999 ± 0.001	5.72 ± 0.01	0.34 ± 0.11
	40%	31.4 ± 3.3	0.161 ± 0.013	0.959 ± 0.007	4.38 ± 0.04	1.49 ± 0.27
	60%	0.2 ± 0.4	0.003 ± 0.000	0.964 ± 0.000	3.21 ± 0.00	2.21 ± 0.15
	80%	0.0 ± 0.0	n/a	n/a	n/a	1.85 ± 0.16
Learned-only	20%	49.6 ± 0.5	0.023 ± 0.015	0.988 ± 0.009	6.05 ± 0.30	0.39 ± 0.12
	40%	48.4 ± 1.0	0.211 ± 0.014	0.892 ± 0.012	6.16 ± 0.25	1.02 ± 0.10
	60%	36.4 ± 3.3	0.620 ± 0.068	0.569 ± 0.048	7.49 ± 0.92	2.07 ± 0.12
	80%	11.2 ± 3.7	0.839 ± 0.039	0.344 ± 0.019	4.73 ± 0.55	1.94 ± 0.10
Heuristic-only	20%	49.8 ± 0.4	0.006 ± 0.002	1.000 ± 0.000	5.72 ± 0.01	0.23 ± 0.02
	40%	44.0 ± 1.3	0.078 ± 0.017	0.998 ± 0.000	4.36 ± 0.02	0.34 ± 0.03
	60%	0.8 ± 0.7	0.110 ± 0.081	0.977 ± 0.009	2.87 ± 0.26	0.78 ± 0.06
	80%	0.0 ± 0.0	n/a	n/a	n/a	0.55 ± 0.03

Table 5: Comparison between HAC (5-seed means) and R2C's published numbers (Peng et al., 2026). At 80% failure, HAC comes within 5.9–7.9 pp of R2C's coverage while omitting subnetwork boundary extraction and multipartite graph construction.

	Random Deployment (REC / COV)			Uniform Deployment (REC / COV)		
Failure	R2C	HAC-Fixed	HAC-Smooth	R2C	HAC-Fixed	HAC-Smooth
20%	0.023 / 0.996	0.051 / 0.997	0.034 / 0.997	0.037 / 1.000	0.005 / 1.000	0.005 / 1.000
40%	0.070 / 0.989	0.134 / 0.988	0.110 / 0.993	0.051 / 0.999	0.040 / 0.998	0.039 / 0.998
60%	0.204 / 0.951	0.331 / 0.928	0.325 / 0.949	0.181 / 0.978	0.249 / 0.960	0.252 / 0.967
80%	0.625 / **0.681**	0.696 / 0.602	0.684 / **0.616**	0.617 / **0.668**	0.643 / **0.609**	0.665 / 0.604

metrics (Peng et al., 2026). From 20% to 60% failure, HAC-Smooth matches R2C's coverage within 0.1 to 1.1 pp and achieves faster recovery at 20% and 40% uniform failure. At 80% failure, R2C's reported coverage exceeds HAC-Fixed by 7.9 pp under random deployment (0.681 vs. 0.602, or 88.4% retention) and by 5.9 pp under uniform deployment (0.668 vs. 0.609, or 91.2% retention), while HAC-Smooth narrows the random-deployment gap to 6.5 pp (0.616, or 90.5% retention). Furthermore, because HAC avoids Quickhull boundary extraction and multipartite graph construction at every step, complete 200-UAV recovery rollouts execute in 0.23–1.29 s (Tables 3–4), or roughly 3.1 ms per step on a single GPU, 32% to 83% faster than Learned-only and SAC-Fixed rollouts.

8.3 Topology Quality Analysis

HAC prevents GNN-induced spatial clustering while sustaining full-swarm reconnection. Figure 1 (right) plots the five-seed trajectory-averaged node degree (AND) across failure rates under random deployment. Whereas GAC-Fixed and Learned-only produce heavily clustered topologies (9.07–13.68 neighbors), HAC-Fixed and HAC-Smooth track the low, smoothly decreasing degree

Table 6: Five-seed schedule steepness sweep under random deployment (mean ± std over 5 seeds), ordered by Co-Active Residual Window CRW(k, s_0). Recovery success at 80% failure increases strictly monotonically with CRW.

Schedule (k, s_0)	CRW	AE(0.8)	40% Succ	60% Succ	80% Succ	80% COV	80% AND
Delayed Steep $(30, 0.5)$	7.5%	45.0%	49.6 ± 0.5	42.8 ± 0.7	5.2 ± 1.3	0.516 ± 0.020	2.60 ± 0.15
HAC-Steep [R2C Default] $(30, 0.125)$	10.0%	82.5%	47.2 ± 1.6	41.0 ± 3.3	14.2 ± 4.3	0.413 ± 0.032	4.11 ± 0.30
Intermediate $(10, 0.125)$	27.5%	72.5%	49.4 ± 0.8	46.0 ± 2.2	18.4 ± 5.0	0.462 ± 0.036	3.43 ± 0.32
Delayed Smooth $(10, 0.5)$	30.0%	35.0%	50.0 ± 0.0	49.8 ± 0.4	29.6 ± 4.5	0.571 ± 0.022	2.58 ± 0.08
HAC-Smooth $(5, 0.7)$	**57.5%**	**0.0%**	$\mathbf{50.0 \pm 0.0}$	$\mathbf{49.6 \pm 0.5}$	$\mathbf{43.6 \pm 2.8}$	$\mathbf{0.616 \pm 0.006}$	$\mathbf{2.48 \pm 0.06}$

profile of the non-clustering Heuristic-only and SAC-Fixed baselines (6.63–6.73 down to 2.48–2.58), confirming that velocity-level virtual-force repulsion counteracts GNN aggregation collapse throughout the recovery trajectory.

9 Ablation Studies

9.1 Branch Contribution Synergy

Velocity fusion yields super-additive recovery gains that exceed the sum of isolated branches at severe failure rates. Comparing HAC-Fixed against its isolated constituents in Tables 3 and 4 rules out simple branch selection. At 60% uniform failure, Heuristic-only (0.8) and Learned-only (36.4) sum to 37.2 recoveries, whereas HAC-Fixed and HAC-Smooth recover in all 50.0 trials. At 80% random failure, Heuristic-only (0.0) and Learned-only (8.6) sum to 8.6 recoveries, yet fusing them reaches 46.6 ± 3.9 (HAC-Fixed) and 47.4 ± 1.9 (HAC-Curr). The global GCN pulls distant components into communication range R_c, where the virtual-force branch's attractive zone (Proposition 1) locks links in place while its repulsive zone prevents over-concentration.

9.2 Schedule Steepness Sweep and Three-Zone Verification

Across five seeds, 80%-failure recovery correlates monotonically with the Co-Active Residual Window CRW **(Spearman $\rho = 1.0$).** Table 6 reports the five-seed schedule sweep ordered by the Co-Active Residual Window CRW(k, s_0) alongside High-Autonomy Exposure AE(k, s_0) from Definition 1. The three-zone decomposition resolves why Delayed Steep $(30, 0.5)$ performs worst at 80% failure (5.2 ± 1.3 recoveries) despite having lower AE than R2C's default $(30, 0.125)$ (45.0% vs. 82.5%): with $k = 30$ and $s_0 = 0.5$, the swarm spends the first 47.5% of the horizon stalled in the Heuristic-Dominated Zone ($w_T < 0.2$, where disconnected components separated beyond R_c cannot move toward each other), passes through a tiny co-active window of CRW = 7.5%, and then switches abruptly into the Unassisted High-Autonomy Zone (AE = 45.0%). Expanding CRW from 7.5% → 10.0% → 27.5% → 30.0% → 57.5% monotonically raises 80%-failure recovery from 5.2 → 14.2 → 18.4 → 29.6 → 43.6 out of 50 trials while monotonically increasing coverage (0.413 → 0.616) and reducing node clustering (AND 4.11 → 2.48).

9.3 Orthogonal 3×3 Train–Evaluation Schedule Decoupling

A complete 3×3 decoupling matrix proves that steep schedules fail at deployment time, whereas time-varying training schedules act as variance-reducing fusion-ratio augmentation. Table 7 evaluates all nine combinations of training schedules $w_T^{\text{train}} \in \{\text{Steep}, \text{Smooth}, \text{Fixed}\}$ and evaluation schedules $w_T^{\text{eval}} \in \{\text{Steep}, \text{Smooth}, \text{Fixed}\}$ across five seeds at 80% failure. Three structural laws emerge across the matrix:

Table 7: Complete 3×3 orthogonal train–evaluation schedule decoupling matrix at 80% failure under random deployment (mean ± std over 5 seeds, 50 trials per seed). Rows vary the training schedule w_T^{train}; columns vary the evaluation schedule w_T^{eval}.

Training Schedule w_T^{train}	Evaluation Schedule w_T^{eval}	80% Success/50	80% REC	80% COV	80% AND
Fixed (0.5)	Steep $(30, 0.125)$	9.8 ± 3.8	0.768 ± 0.070	0.439 ± 0.054	3.83 ± 0.51
	Smooth $(5, 0.7)$	42.6 ± 5.0	0.702 ± 0.042	0.613 ± 0.008	2.48 ± 0.03
	Fixed (0.5) [**HAC-Fixed**]	46.6 ± 3.9	0.696 ± 0.048	0.602 ± 0.009	2.58 ± 0.03
Smooth $(5, 0.7)$	Steep $(30, 0.125)$	12.0 ± 4.6	0.797 ± 0.037	0.410 ± 0.027	4.06 ± 0.22
	Smooth $(5, 0.7)$ [**HAC-Smooth**]	43.6 ± 2.8	0.684 ± 0.020	0.616 ± 0.006	2.48 ± 0.06
	Fixed (0.5) [**HAC-Curr**]	$\mathbf{47.4 \pm 1.9}$	$\mathbf{0.674 \pm 0.027}$	0.602 ± 0.005	2.58 ± 0.03
Steep $(30, 0.125)$	Steep $(30, 0.125)$ [**HAC-Steep**]	14.2 ± 4.3	0.796 ± 0.032	0.413 ± 0.032	4.11 ± 0.30
	Smooth $(5, 0.7)$	46.4 ± 0.5	0.706 ± 0.018	$\mathbf{0.624 \pm 0.003}$	$\mathbf{2.47 \pm 0.04}$
	Fixed (0.5)	46.6 ± 0.5	0.679 ± 0.023	0.622 ± 0.004	2.51 ± 0.05

Table 8: Five-seed counter-experiment with HSC (heuristic branch + subnetwork-level rigid-translation GCN, mean ± std over 5 seeds). Schedule smoothing rescues 60%-failure recovery but cannot overcome HSC's lack of node-level expressiveness at 80% failure.

	Random Deployment (Success/50)			Uniform Deployment (Success/50)		
Variant	40%	60%	80%	40%	60%	80%
HSC-Fixed	47.6 ± 1.5	38.0 ± 2.7	0.4 ± 0.5	49.6 ± 0.5	$\mathbf{46.4 \pm 1.4}$	0.2 ± 0.4
HSC-Steep $(30, 0.125)$	47.2 ± 1.5	12.2 ± 3.7	0.0 ± 0.0	49.4 ± 0.5	13.0 ± 4.9	0.0 ± 0.0
HSC-Smooth $(5, 0.7)$	$\mathbf{49.2 \pm 0.7}$	$\mathbf{40.8 \pm 1.5}$	0.0 ± 0.0	$\mathbf{49.8 \pm 0.4}$	45.0 ± 2.8	0.0 ± 0.0

1. **Deployment-time steepness is the primary cause of collapse:** Regardless of training regime, evaluating under 'Steep' $(30, 0.125)$ collapses 80%-failure recovery to 9.8–14.2/50 with severe clustering (AND = 3.83–4.11), because $w_E^{\text{eval}}(t)$ vanishes before reconnection completes.
2. **Restoring test-time heuristic weight rescues all trained policies:** Switching w_T^{eval} from 'Steep' to 'Smooth' $(5, 0.7)$ or 'Fixed' (0.5) boosts recovery by +28.4 to +36.8 trials across every row. 'Eval-Smooth' achieves the highest post-recovery coverage (0.613–0.624) and lowest node degree (2.47–2.48), while 'Eval-Fixed' maximizes recovery count (46.6–47.4/50).
3. **Time-varying training schedules cut cross-seed variance by 50% to 87%:** Because time-varying schedules expose the GCN to a continuum of (w_E, w_T) ratios during the 40-step training rollout, 'Train-Smooth' and 'Train-Steep' reduce the 80%-failure success standard deviation under 'Eval-Fixed' from ±3.9 ('Train-Fixed') down to ±1.9 (**HAC-Curr**) and ±0.5, respectively.

9.4 Granularity Substitution: HSC as a Counter-Experiment

Schedule smoothing rescues subnetwork-level hybrids at moderate fragmentation (60% failure) but cannot overcome rigid-translation expressiveness limits at 80% failure. We evaluate **HSC** across five seeds by fusing the virtual-force heuristic branch with SAC's subnetwork-level rigid-translation GCN. As shown in Table 8, at 60% failure, replacing the steep schedule $(30, 0.125)$ with the smooth schedule $(5, 0.7)$ rescues HSC from 12.2 ± 3.7 to 40.8 ± 1.5 recoveries under random deployment (surpassing HSC-Fixed's 38.0 ± 2.7) and from 13.0 ± 4.9 to 45.0 ± 2.8 under uniform deployment. However, at 80% failure, all three HSC variants collapse to 0.0–0.4/50 across both deployments. When multi-node subnetworks remain intact at 60% failure, smooth scheduling restores hybrid coordination, but when 80% node loss shatters the swarm into isolated singletons and tiny clusters, rigid subnetwork translation lacks the node-level degrees of freedom required to reconnect distant components.

10 Discussion

10.1 Underlying Physical and Optimization Mechanisms

Complementary receptive fields, deployment safety margins, and training-time fusion-ratio augmentation explain the effectiveness of simple global hybrids. Across our five-seed studies, the heuristic virtual-force branch and the global k-NN GCN address orthogonal spatial scales: the global GCN supplies long-range attraction gradients across disconnected components separated beyond R_c, while the virtual-force branch enforces local collision buffering ($d < d_l$) and certifiable link capture ($d_u < d \leq R_c$, Proposition 1). Our 3×3 decoupling study (Table 7) demonstrates that maintaining $w_E^{\text{eval}}(t) \geq w_E^{\min}$ throughout deployment prevents the learned branch from breaking newly formed links, while time-varying training schedules expose the GCN to diverse weighting ratios across rollout steps and cut cross-seed variance by more than half.

10.2 Practical Deployment Guidelines

Practitioners should match fusion schedules to policy expressiveness and expected failure severity. Our findings translate into three actionable design rules for resilient multi-UAV controllers:

- **Combine smooth curriculum training with fixed 50/50 evaluation (HAC-Curr) for peak high-damage reliability:** Training with ($k = 5, s_0 = 0.7$) and deploying with $w_E = w_T = 0.5$ achieves 100% recovery across 20%–60% failures and the highest 80%-failure recovery rate ($47.4 \pm 1.9/50$).
- **Deploy smooth evaluation schedules** ($k = 5, s_0 = 0.7$) **when post-recovery coverage uniformity is paramount:** Evaluating under $(5, 0.7)$ achieves faster recovery at light-to-moderate damage and maximizes 80%-failure coverage (0.616–0.624 vs. 0.602 for fixed evaluation) with minimal node clustering (AND 2.47–2.48).
- **Ensure node-level output expressiveness before expecting resilience above 60% node loss:** Subnetwork-level rigid translation (SAC, HSC) can be stabilized by smooth scheduling up to 60% failure (40.8–45.0/50), but requires per-node velocity residuals to survive 80% fragmentation.

10.3 Limitations and Future Work

Closing the remaining 5.9 to 7.9 pp high-failure coverage gap and scaling training horizons remain open directions. First, while HAC retains roughly 90% of R2C's reported coverage at 80% failure, isolating whether R2C's remaining 5.9–7.9 pp advantage stems from Quickhull boundary weighting, multipartite virtual edges, or its 16× wider spectral GCN requires a controlled re-implementation of those sub-modules inside our simulator. Second, all learned policies are trained on $T_{\text{train}} = 40$ rollout steps for backpropagation-through-time tractability and evaluated over $T = 400$ steps; curriculum horizon scaling over longer rollouts could further improve late-stage coverage.

11 Conclusion

We systematically deconstructed the hybrid design space for UAV swarm connectivity recovery along decision paradigm, spatial reasoning granularity, and trade-off scheduling across five random seeds. By pairing a parameter-free virtual-force branch with a single global k-NN graph convolution branch, HAC achieves near-perfect connectivity restoration across 20% to 80% node failures, outperforms single-branch learned baselines by up to 76.4 pp in recovery rate, executes in 3.1 ms per step, and retains 88.4% to 91.2% of R2C's reported coverage without subnetwork partitioning or convex-hull

boundary extraction. Through five-seed schedule steepness sweeps, a complete 3×3 train–evaluation decoupling matrix, and a subnetwork-level counter-experiment (HSC), we proved that expanding the Co-Active Residual Window preserves local heuristic safety margins at deployment time while regularizing end-to-end rollout training, provided the learned policy possesses node-level velocity expressiveness.

References

Ameer A. Abbasi and Mohamed Younis. Movement-assisted connectivity restoration in wireless sensor and actor networks. *IEEE Transactions on Parallel and Distributed Systems*, 20(9):1366–1379, 2008.

Kemal Akkaya, Fatih Senel, Aravind Thimmapuram, and Suleyman Uludag. Distributed recovery from network partitioning in movable sensor/actor networks via controlled mobility. *IEEE Transactions on Computers*, 59(2):258–271, 2009.

C. Bradford Barber, David P. Dobkin, and Hannu Huhdanpaa. The quickhull algorithm for convex hulls. *ACM Transactions on Mathematical Software*, 22(4):469–483, 1996.

Ming Chen, Hao Wang, Chih-Yung Chang, and Xin Wei. SIDR: A swarm intelligence-based damage-resilient mechanism for UAV swarm networks. *IEEE Access*, 8:77089–77105, 2020.

Andrea Coletta, Francesco Giorgi, Gaia Maselli, Marco Prata, Daniel Silvestri, Jonathan Ashdown, and Francesco Restuccia. A 2-UAV: Application-aware content and network optimization of edge-assisted UAV systems. In *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, pages 1–10, 2023.

Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018.

Zhiwei Guo, Yifan Sun, Yue Wang, Chen Fu, and Jing Zhong. An intelligent framework for resilience recovery of FANETs with spatio-temporal aggregation and multi-head attention mechanism. *Computers, Materials & Continua*, 79(2):2375–2398, 2024.

Yogesh K. Joshi. Autonomous recovery from multi-node failure in wireless sensor networks. In *2012 IEEE Global Communications Conference (GLOBECOM)*, pages 652–657, 2012.

Nikolaos Karalias and Andreas Loukas. Erdos goes neural: An unsupervised learning framework for combinatorial optimization on graphs. *Advances in Neural Information Processing Systems*, 33: 6659–6672, 2020.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

Harrshavasan Kurunathan, Hailong Huang, Kai Li, Wei Ni, and Ekram Hossain. Machine learning-aided operations and communications of unmanned aerial vehicles: A contemporary survey. *IEEE Communications Surveys & Tutorials*, 26(1):496–533, 2023.

Jian Li, Ping Yi, Tong Duan, Ze Zhang, Jing Li, Yue Wang, and Jian Yu. Fast connectivity restoration of UAV communication networks based on distributed hybrid MADDPG and APF algorithm. *Ad Hoc Networks*, 171:103785, 2025.

Jun Li, Cheng Zhou, Jia Liu, Min Sheng, Nan Zhao, and Yi Su. Reinforcement learning-based resource allocation for coverage continuity in high dynamic UAV communication networks. *IEEE Transactions on Wireless Communications*, 23(2):848–860, 2023.

Hao Lin and Liang Ding. Multi-hop differential topology based algorithms for resilient network of UAV swarm. *arXiv preprint arXiv:2411.11342v2*, 2025.

Hao Lin, Liang Ding, Song Chen, Fan Yang, and Ling Qian. Multi-hop diffused graph convolution for resilient UAV swarm networks. In *2024 IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB)*, pages 1–6, 2024.

Fang Liu and Qing Wang. Asymmetric learning for spectral graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 18798–18806, 2025.

Zhangfeng Ma, Bo Ai, Ruisi He, Guangjie Wang, Yong Niu, and Zhangdui Zhong. A wideband non-stationary air-to-air channel model for UAV communications. *IEEE Transactions on Vehicular Technology*, 69(2):1214–1226, 2019.

Zhenqiang Mi and Jenny Y. Yang. Obstacle-avoidance connectivity restoration for mobile sensor systems with local information. In *2015 IEEE International Conference on Communications (ICC)*, pages 6395–6399, 2015.

Zhenqiang Mi, Yang Yang, and Guangming Liu. HERO: A hybrid connectivity restoration framework for mobile multi-agent networks. In *2011 IEEE International Conference on Robotics and Automation*, pages 1702–1707, 2011.

Zhiyu Mou, Feifei Gao, Jian Liu, and Qing Wu. Resilient UAV swarm communications with graph convolutional neural network. *IEEE Journal on Selected Areas in Communications*, 40(1):393–411, 2022.

Dor Mutzari, Yonatan Aumann, and Sarit Kraus. Heterogeneous multi-robot graph coverage with proximity and movement constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 14646–14654, 2025.

Hao Ni, Qiuming Zhu, Bo Hua, Kai Mao, Yuxin Pan, Farman Ali, Weizhi Zhong, and Xiaomin Chen. Path loss and shadowing for UAV-to-ground UWB channels incorporating the effects of built-up areas and airframe. *IEEE Transactions on Intelligent Transportation Systems*, 25(11):17066–17077, 2024.

Yabin Peng, Chang Liu, Shaoxun Liu, Yuan Liu, and Yue Wu. SmartTRO: Optimizing topology robustness for internet of things via deep reinforcement learning with graph convolutional networks. *Computer Networks*, 218:109385, 2022.

Yabin Peng, Jun Wu, Tong Duan, Yuan Liu, Ze Zhang, and Jie Zhang. Prioritized recovery strategy for robust UAV swarm communication via graph reinforcement learning. *IEEE Internet of Things Journal*, 12(13):23891–23904, 2025.

Yabin Peng, Chenyu Zhou, Hainan Cui, Tong Duan, Haoyang Chen, Fan Zhang, and Shaoxun Liu. Resilient UAV swarm with fast connectivity recovery and extensive coverage. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 917–925, 2026.

Jan Rychlewski. On hooke's law. *Journal of Applied Mathematics and Mechanics*, 48(3):303–314, 1984.

Ran Shi, Xin Yu, Yue Wang, Ye Tian, Zhi Liu, Wei Wu, Xiao-Ping Zhang, and Manuela M. Veloso. Symmetry-informed MARL: A decentralized and cooperative UAV swarm control approach for communication coverage. *IEEE Transactions on Mobile Computing*, 24(9):8039–8056, 2025.

Siddharth Shriwastav. Round-table negotiation for fast restoration of connectivity in partitioned wireless sensor networks. *Ad Hoc Networks*, 77:11–27, 2018.

Mehmet Tosun, Umut Can Cabuk, Emre Haytaoglu, Orhan Dagdeviren, and Yusuf Ozturk. DP-kCR: Distributed proactive k-connectivity recovery algorithm for UAV-based MANETs. *IEEE Transactions on Reliability*, 73(4):1918–1932, 2024.

Jian Wang, Shuo Li, Di Chen, Qiang Fu, Qi Qi, Hao Sun, Bo He, and Jianxin Liao. Flight trajectory control with network-oriented hierarchical reinforcement learning for UAV-assisted time-sensitive IoT. *IEEE Transactions on Intelligent Transportation Systems*, 26(5):6332–6345, 2025a.

Xu Wang, Zhen Zhao, Liang Yi, Zhaolong Ning, Lei Guo, F. Richard Yu, and Song Guo. A survey on security of UAV swarm networks: Attacks and countermeasures. *ACM Computing Surveys*, 57 (3):1–37, 2024a.

Yao Wang, Wei Chen, Tom H. Luan, Zhou Su, Qichao Xu, Ru Li, and Ning Chen. Task offloading for post-disaster rescue in unmanned aerial vehicle networks. *IEEE/ACM Transactions on Networking*, 30(4):1525–1539, 2022.

Yingze Wang, Zhen Feng, Hui Zhang, Yang Gao, Jing Lei, Le Sun, and Ming Song. Angle robustness unmanned aerial vehicle navigation in GNSS-denied scenarios. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 10386–10394, 2024b.

Yue Wang, Xu Wang, Chao Wei, Qi Ren, and Yong Tang. UAV swarm network topology self-healing via graph-based deep reinforcement learning. In *2025 IEEE Wireless Communications and Networking Conference (WCNC)*, pages 1–6, 2025b.

Zhi Wang, Xin Zhao, Jie Zhang, Nan Yang, Peng Wang, Jian Tang, Jun Zhang, and Liang Shi. APF-CPP: An artificial potential field based multi-robot online coverage path planning approach. *IEEE Robotics and Automation Letters*, 9(11):9199–9206, 2024c.

Chejian Xu, Wenhao Ding, Weijie Lyu, Zuxin Liu, Shuai Wang, Yihan He, Hanjiang Hu, Ding Zhao, and Bo Li. SafeBench: A benchmarking platform for safety evaluation of autonomous vehicles. *Advances in Neural Information Processing Systems*, 35:25667–25682, 2022.

Zhiqiang Ye, Ke Wang, Yu Chen, Xiang Jiang, and Guanghua Song. Multi-UAV navigation for partially observable communication coverage by graph reinforcement learning. *IEEE Transactions on Mobile Computing*, 22(7):4056–4069, 2022.

Dingyi Zeng, Wanlu Liu, Wenjie Chen, Liang Zhou, Miao Zhang, and Hua Qu. Substructure aware graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11129–11137, 2023.

Lei Zhang, Yi Du, Jing Xu, and Xin Wang. UAV-enabled IoT: Cascading failure model and topology-control-based recovery scheme. *IEEE Internet of Things Journal*, 11(12):22562–22577, 2024.

A Deviations and Bug-Fix Log

We document every implementation choice where our testbed deviates from Peng et al. (2026) or corrects numerical issues discovered during development:

- **Failure-node selection.** R2C specifies "unpredictable random failure patterns" without detailing the sampling distribution; we apply uniform random node removal without replacement.
- **Count-invariant connectivity surrogate loss.** R2C's discrete component count objective is non-differentiable. An initial soft-minimum estimator omitted the $-\log(n_i n_j)$ normalization in Eq. (5), causing large component pairs to produce spuriously negative losses purely due to pair-count summation bias. On a synthetic test with true minimum separation $\approx$ 100–108 m, the uncorrected estimator returned 2.85 for 3×3 nodes vs. -230.58 for 30×30 nodes, whereas our corrected Eq. (5) returned 112.71 and 109.54, respectively. All reported models use the corrected loss.
- **Boundary buffer width and clamping.** R2C leaves d_e unspecified; we set $d_e = 0.15R_c = 18$ m and enforce $0 \leq d_{i,b} < d_e$ so nodes clamped exactly onto mission boundaries continue to receive inward repulsion.
- **Rollout horizon and Monte Carlo coverage.** Differentiable training uses $T_{\text{train}} = 40$ steps with 800 uniform random grid points per step, whereas evaluation runs for $T = 400$ steps on a deterministic 5 m grid (40,000 points).
- **Full-stack training and evaluation seeding.** Both NumPy environment RNGs and PyTorch CPU/CUDA weight initializations are seeded via `-seed` in both training and evaluation scripts. Earlier unseeded GAC-Fixed runs yielded an optimistic 20%-failure COV of 0.830 and AND of 11.87, whereas five properly seeded runs yield COV 0.747 ± 0.009 and AND 13.68 ± 0.28 (Table 3). All tables in this paper report five-seed means and standard deviations under strict full-stack seeding.

B Hyperparameter Summary

Table 9: Hyperparameters shared across all trained architectures unless otherwise noted.

Parameter	Value
GCN layers L_{gcn} / Hidden dimension	4 / 128
Euclidean k-NN degree k_{nn}	8
Optimizer / Learning rate / Gradient clip	Adam / 1×10^{-3} / 1.0
Training epochs / Training rollout T_{train}	300 / 40 steps
Evaluation horizon T / Step size Δt	400 steps / 1.0 s
Movement cost coefficient λ	0.01
Soft-min temperatures $\tau_{\text{c}}, \tau_{\text{v}}$	50.0 m, 10.0 m
Heuristic decay constants α, β, δ	0.01
Neutral zone $[d_l, d_u]$ / Boundary buffer d_e	$[0.8R_c, 0.9R_c]$ / $0.15R_c$
Communication & coverage radii $R_c = R_{\text{cov}}$	120 m
Maximum UAV speed $\hat{v}_{\max}$	1.0 m/s
Mission area $\mathcal{A}$	1000×1000 m^2
Swarm size (Random / Uniform 14×14)	200 / 196 UAVs
Independent random seeds $\times$ Eval trials per seed	$5 \times 50 = 250$ trials

C Code Structure and Full Reproduction Suite

The accompanying repository provides end-to-end training, evaluation, and five-seed aggregation scripts for all architectures and ablations reported in Tables 2–8, together with the aggregated five-seed summary CSVs in `code/results/summary_multiseed_all.csv`.